

ÁRBOL BINARIO · ELIMINACIÓN

1

Poda de Hijos Únicos

10 puntos

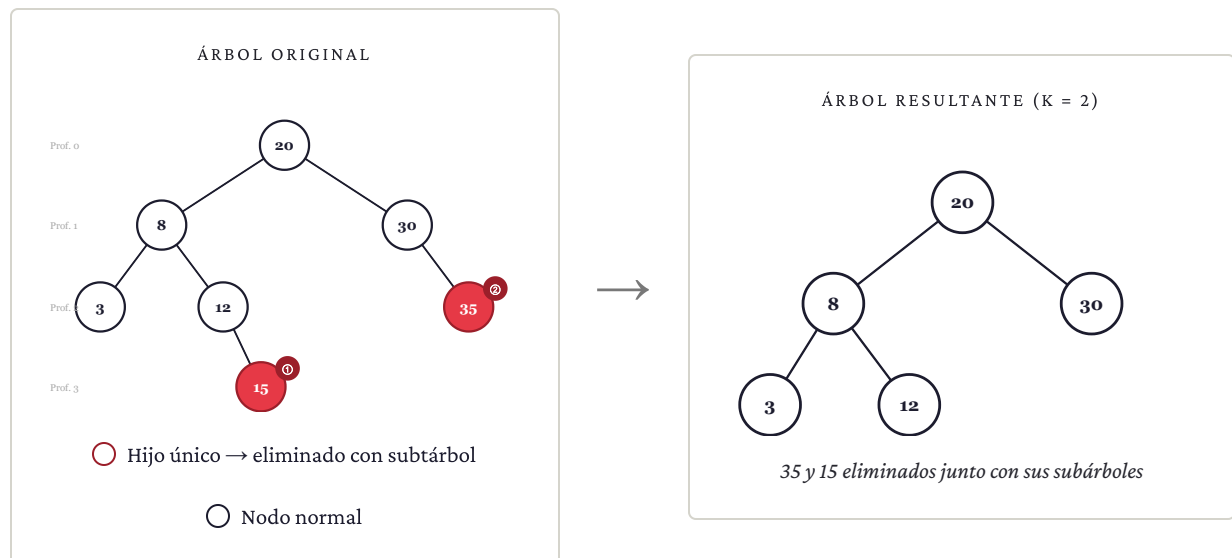
Dado un árbol binario y un entero k , elimine los k primeros nodos hijo único encontrados en un recorrido en **preorden de izquierda a derecha** (raíz $\rightarrow$ izquierda $\rightarrow$ derecha, por profundidad). Un nodo es *hijo único* si su padre tiene *exactamente un hijo*. Al eliminar un nodo hijo único, se elimina también todo su subárbol. Implemente la función:

```
void eliminarHijosUnicos(NodoBinario* raiz, int k)
```

RESTRICCIONES

 $1 \leq k \leq n$ $n \geq 1$ Recorrido DFS preorden — raíz $\rightarrow$ izq $\rightarrow$ der (por profundidad)Si hay menos de k hijos únicos, elimine todos los que haya

Libere la memoria de cada nodo eliminado

EJEMPLO DE EJECUCIÓN — $K = 2$ 

Recorrido DFS preorden: $20 \rightarrow 8 \rightarrow 3 \rightarrow 12 \rightarrow 15 \rightarrow 30 \rightarrow 35$. El nodo 15 es el único hijo de 12 y se encuentra primero en profundidad (eliminado — ①). El nodo 35 es el único hijo de 30 y se encuentra después al retroceder hacia la rama derecha (eliminado — ②). Los nodos 3, 12, 8 no son hijos únicos porque sus padres tienen dos hijos. Con $k = 2$ se eliminan 15 y 35 junto con sus respectivos subárboles.

Duplicación de Hijos

Dado un árbol general y un entero k , identifique todos los nodos que tengan **exactamente k hijos** y agréguele a cada uno **k nuevos hijos** cuyo valor sea igual al valor del nodo padre. El recorrido debe hacerse en **preorden**. Implemente la función:

```
void duplicarHijos(NodoGeneral* raiz, int k)
```

RESTRICCIONES

$$1 \leq k \leq n$$

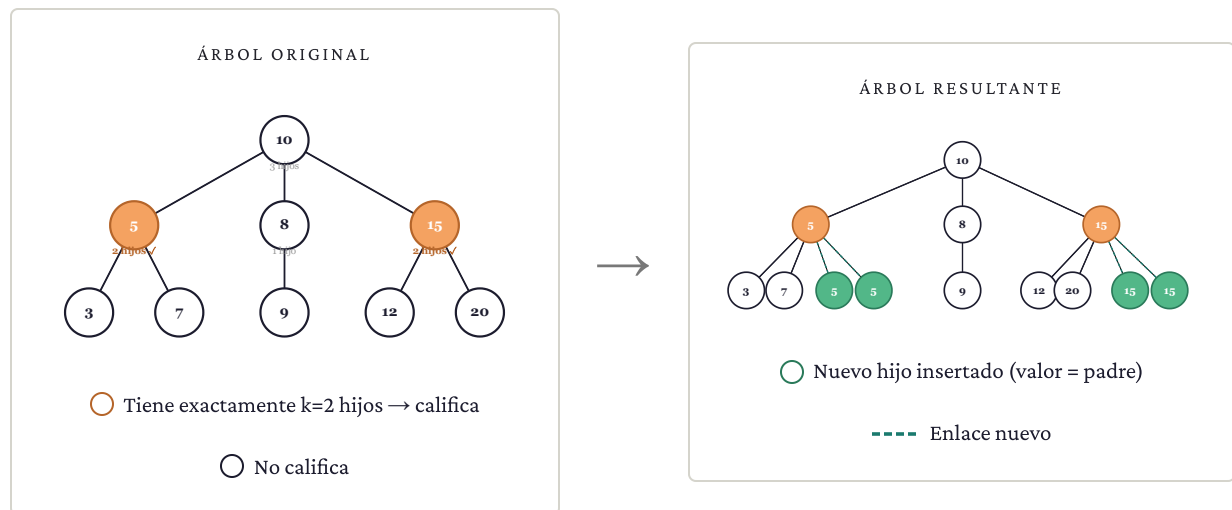
$$n \geq 1$$

Los nuevos hijos se agregan al final de la lista de hijos existentes

Los nodos recién insertados NO se procesan en el mismo recorrido

Valor del nuevo hijo = valor del padre que lo genera

EJEMPLO DE EJECUCIÓN — $k = 2$



Nodos calificados ($k = 2$): 5 (hijos: 3, 7) y 15 (hijos: 12, 20). El nodo 10 tiene 3 hijos y el 8 tiene 1, por lo que no califican. En preorden, 5 se procesa antes que 15. A cada calificado se le agregan al final 2 nuevos hijos con su propio valor. Los nodos recién creados no se visitan durante el mismo recorrido.

BST · BÚSQUEDA Y RECORRIDO

3

Menores en Orden Descendente

10 puntos

Dado un árbol binario de búsqueda (BST) y un entero k , retorne una lista con todos los nodos cuyo valor sea **estrictamente menor que k** , ordenados de **mayor a menor**. Si el valor k **no existe en el BST**, retorne la lista `[-1]`. Implemente la función:

```
Lista* menoresDescendente(NodoBST* raiz, int k)
```

RESTRICCIONES

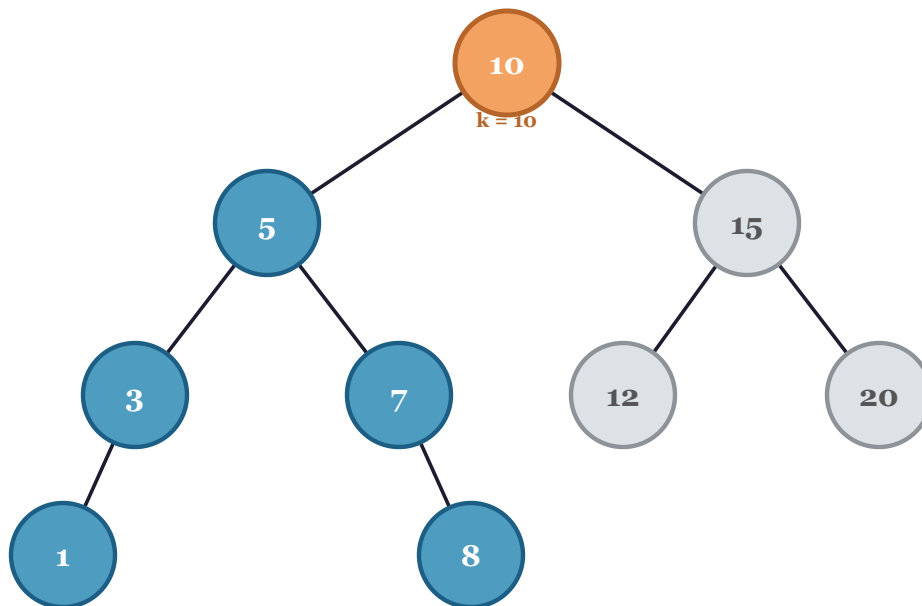
El árbol cumple estrictamente la propiedad BST

Sin valores repetidos

 k no existe en el BST → retornar `[-1]` k existe pero no hay menores → retornar `[]`

Lista ordenada de mayor a menor

Lograr la solución en un solo recorrido

EJEMPLO DE EJECUCIÓN — $k = 10$ (EXISTE EN EL BST)BST — NODOS MENORES A $k = 10$ RESALTADOS EN AZUL

○ $k = 10$ — valor buscado ● Menor que k → incluido en resultado ○ Mayor que k → ignorado

RESULTADO — MENORES A 10, DE MAYOR A MENOR `[ 8, 7, 5, 3, 1 ]`

CASO — $K = 6$ (NO EXISTE EN EL BST) [-1]